VictoriaBank Business IPS Integration API

API Name: IPS Business WebApi

Version: 2.0.18

Base URL: https://test-ipspj.victoriabank.md

Swagger: https://test-ipspj.victoriabank.md/index.html

Introduction

This RESTful API integrates juridical systems with BNM IPS. It allows juridical persons to connect their systems, such as online shops, invoicing systems, and other e-commerce platforms, to the Instant Payment System (IPS) provided by the National Bank of Moldova (BNM). With this integration, businesses can efficiently handle real-time payments, Request to Pay (RTP) for invoices, and QR codes for online payments and other transaction-related operations.

Abbreviations

- CBS: Core Banking System
- CAS: Central Aliases Services
- IBAN: International Bank Account Number
- IPS: Instant Payments System
- P2P: Peer to Peer
- RTP: Request to Pay

Authentication

Our RESTful API utilizes JWT (JSON Web Tokens) for authentication. When integrating with this API, developers need to ensure correct token handling:

- **Headers**:
    - **Authorization: Bearer <token>**

The process for obtaining the JWT token is detailed in the additional documentation.

Every API request mandates the inclusion of this header for successful authentication. Ensure that the header is present and valid for each request. The API operates exclusively over HTTPS, prioritizing data safety.

Error Handling:

Clients should be prepared to handle authentication errors. Possible error responses related to authentication include:

- 401 Unauthorized:
    - Absence of the Authorization header.
    - Invalid or expired JWT token.

HTTP Status Codes

- Status Code: 200 Success
- Status Code: 5xx Server error

API Errors

Returned as JSON.

Example:

```
{
  "traceReference": "a19df424-942a-4124-ae6b-25284ad5abfb",
  "errorCode": "VB10403",
  "description": "IDX10230: Lifetime validation failed. Delegate returned false,
securitytoken: '\[PII of type 'Microsoft.IdentityModel.JsonWebTokens.JsonWebToken' is
hidden. For more details, see <https://aka.ms/IdentityModel/PII.\>]'."
}
```

Error messages

In the attachment, you can find the list of error messages from BNM related to QR code generation. The error code is composed of Severity, Facility, and Error Code (e.g., EQ1). Error messages from VictoriaBank will come with the VB prefix.

File : *errors-and-warnings-qr.pdf*

Endpoints

Authorization

Overview

The Authorization Endpoint is used to obtain and refresh JWT tokens required for authenticating API requests. This endpoint supports two grant types:

- **Password Grant**: Obtain an access token using username and password.
- **Refresh Token Grant**: Obtain a new access token using a refresh token.

Endpoint Details

URL: api/identity/token

Method: POST

Content-Type: application/x-www-form-urlencoded

Grant Types

- **Password Grant**
    - **Description**: Obtain an access token using user credentials.
    - **cURL Example**:

```
curl --location 'https://…/identity/token' \
     --header 'Content-Type: application/x-www-form-urlencoded' \
     --data-urlencode 'grant_type=password' \
     --data-urlencode 'username=string' \
     --data-urlencode 'password=string'
```

- **Request Parameters:**
  - **grant_type** (required): Must be set to *password*.
  - **username** (required): The username of the client.
  - **password** (required): The password of the client**.**
- **Response Body:**

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tokenType": "bearer",
  "expiresIn": 3600,
  "refreshToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshExpiresIn": 18000
}
```

- **Refresh Token Grant**
  - **Description**: Obtain a new access token using a refresh token.
  - **cURL Example**:

```
curl --location 'https://…/identity/token' \
     --header 'Content-Type: application/x-www-form-urlencoded' \
     --data-urlencode 'grant_type=refresh_token' \
     --data-urlencode 'refresh_token=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...'
```

- **Request Parameters:**
  - **grant_type** (required): Must be set to *refresh_token*.
  - **refresh_token** (required): The refresh token obtained from a previous authentication request.
- **Response Body:**

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tokenType": "bearer",
  "expiresIn": 3600,
  "refreshToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshExpiresIn": 18000
}
```

Response Details

- **accessToken**: The JWT access token to be used for authenticated API requests.
- **tokenType**: The type of the token, typically bearer.
- **expiresIn**: The duration in seconds for which the access token is valid (e.g., 3600 seconds).
- **refreshToken**: The JWT refresh token used to obtain a new access token.
- **refreshExpiresIn**: The duration in seconds for which the refresh token is valid (e.g., 18000 seconds).

Notes

- Ensure that all requests to the Authorization Endpoint are made over HTTPS to maintain security.
- Store refresh tokens securely, as they can be used to obtain new access tokens.
- Handle token expiration gracefully by implementing token refresh logic in your application.

QR

QR Code Types and Amount Types

This part provides details on various QR code types and the associated amount types, along with examples.

QR Code Types

- **DYNM**: Every QR is unique and can be used only for one transaction; it is destroyed afterward.
- **STAT**: Generates a QR that can be used repeatedly.
  Note: A STAT QR is valid only for 90 days from the last payment.
- **HYBR**: The QR is static, but the transaction information contained in it is different.

QR Type Table

	DYNM	STAT	HYBR
Required TTL	✅	❌	✅

Amount Types

- **Fixed**: Use if you want the client to pay a fixed sum.
- **Controlled**: Use if you want the client to pay an amount within a defined range.
- **Free**: Use if you want the client to pay any amount they wish.

Amount Type Table

	Fixed	Controlled	Free
amount.sum	✅	❌	❌
amountMin / amountMax	❌	✅	❌

Examples

Dynamic QR with Fixed Amount

```
{
  "header": {
    "qrType": "DYNM",
    "amountType": "Fixed",
    "pmtContext": "e"
  },
  "extension": {
    "creditorAccount": {
      "iban": "XXX"
    },
    "amount": {
      "sum": 100.12,
      "currency": "MDL"
    },
    "dba": "string",
    "remittanceInfo4Payer": "string",
    "creditorRef": "string",
    "ttl": {
      "length": 360,
      "units": "mm"
    }
  }
}
```

Static QR with Free Amount Type

```
{
  "header": {
    "qrType": "STAT",
    "amountType": "Free",
    "pmtContext": "e"
  },
"extension": {
    "creditorAccount": {
      "iban": "MD52VI225100000103171MDL"
    },
    "dba": "string",
    "remittanceInfo4Payer": "string",
    "creditorRef": "Some content here"
  }
}
```

Static QR with Controlled Amount Type

```
{
  "header": {
    "qrType": "STAT",
    "amountType": "Controlled",
```

```
    "pmtContext": "e"
  },
"extension": {
    "creditorAccount": {
      "iban": "MD52VI225100000103171MDL"
    },
    "amountMin": {
      "sum": "1",
      "currency": "MDL"
    },
    "amountMax": {
      "sum": "10",
      "currency": "MDL"
    },
    "dba": "string",
    "remittanceInfo4Payer": "string",
    "creditorRef": "Some content here"
  }
}
```

New QR

URL: /api/v1/qr

Method: POST

Description: Register new payee-presented QR code

Parameters:

- **width -** integer($int32) : used to request a QR code image with specific width(Default:300)
- **height** - integer($int32) : used to request a QR code image with specific height (Default:300)

Request body:

```
{
  "header": {
    "qrType": "DYNM",  // Type of QR code: DYNM - Dynamic QR, STAT - Static QR, HYBR -
Hybrid QR
    "amountType": "Fixed",  // Specifies the type of amount: Fixed - Dynamic QR, Controlled
- Static QR, Free - Hybrid QR
    "pmtContext": "o"  // Payment context: m - mobile payment, e - e-commerce payment, i -
invoice payment, o - other (is letter)
  },
  "extension": {
    "creditorAccount": {
      "iban": "string"  // The account from which funds will be debited credited.
    },
    "amount": {
      "sum": "string",  // The total sum to be paid.
      "currency": "string"  // The currency in which the sum is specified.
    },
    "amountMin": {
      "sum": "string",  // (Optional) The minimum sum to be paid.
```

```
      "currency": "string"  // (Optional) The currency for the minimum sum.
    },
    "amountMax": {
      "sum": "string",  // (Optional) The maximum sum to be paid.
      "currency": "string"  // (Optional) The currency for the maximum sum.
    },
    "dba": "string",  // Commercial name that will appear in client APP.
    "remittanceInfo4Payer": "string",  // Payment destination.
    "creditorRef": "string",  // External payment reference.
    "ttl": {
      "length": 0,  // The duration for which the QR code is valid.
      "units": "mm"  // The unit of time for the TTL: ss - seconds, mm - minutes
    }
  }
}
```

Response body:

```
{
  "qrHeaderUUID": "string",  // Unique identifier for the QR header.
  "qrExtensionUUID": "string",  // Unique identifier for the QR extension.
  "qrAsText": "string",  // QR code represented as text.
  "qrAsImage": "string"  // QR code represented as an image (base64 encoded).
}
```

Cancel QR

URL: /api/v1/qr/{qrHeaderUUID}

Method: DELETE

Description: Cancel payee-resented QR code, including active extension, if exists

Parameters:

- qrHeaderUUID **–** This is the same UUID that was returned in New QR endpoint.

QR Status

URL: /api/v1/qr/{qrHeaderUUID}/status

Method: GET

Description: Get the status of a payee-presented QR code header, statuses of N last extensions, and list of M last payments against each extension.

Parameters:

- nbOfExt
- **Description**: Limits the number of requested QR code extensions.
- **Type**: integer (numberOfItems) $0 .. 10$
- **Default**: 0
- **Example**: nbOfExt=5

- nbOfTxs
- **Description**: Limits the number of requested transactions (payments).
- **Type**: integer (numberOfItems) $0 .. 10$
- **Default**: 0
- **Example**: nbOfTxs=5

Response body:

```
{
  "uuid": "string",
  "status": "Active", // Enum: "Active", "Paid", "Expired", "Cancelled", "Replaced",
"Inactive"
  "statusDtTm": "2024-07-29T09:59:09.231Z",
  "lockTtl": {
    "length": 0,
    "units": "mm"
  },
  "extensions": [
    {
      "uuid": "string",
      "isLast": true,
      "status": "Active",
      "statusDtTm": "2024-07-29T09:59:09.231Z",
      "isHeaderLocked": true,
      "ttl": {
        "length": 0,
        "units": "ss"
      },
      "payments": [
        {
          "system": "string",
          "reference": "string",
          "amount": {
            "sum": 0,
            "currency": "string"
          }
        }
      ]
    }
  ]
}
```

Extensions

New extension

URL: /api/v1/qr/{qrHeaderUUID}/extentions

Method: POST

Description: Register new extension for HYBR or STAT payee-presented QR code. This extension will replace the previously registered extension.

Parameters:

- qrHeaderUUID – This is the same UUID that was returned in New QR endpoint.

Request body:

```
{
  "creditorAccount": {
    "iban": "string"  // The account from which funds will be debited.
  },
  "amount": {
    "sum": "string",  // The total sum to be paid.
    "currency": "string"  // The currency in which the sum is specified.
  },
  "amountMin": {
    "sum": "string",  // (Optional) The minimum sum to be paid.
    "currency": "string"  // (Optional) The currency for the minimum sum.
  },
  "amountMax": {
    "sum": "string",  // (Optional) The maximum sum to be paid.
    "currency": "string"  // (Optional) The currency for the maximum sum.
  },
  "dba": "string",  // Commercial name that will appear in client APP.
  "remittanceInfo4Payer": "string",  // Payment destination.
  "creditorRef": "string",  // External payment reference.
  "ttl": {
    "length": 0,  // The duration for which the QR code is valid.
    "units": "mm"  // The unit of time for the TTL: ss - seconds, mm - minutes
  }
}
```

Response body:

```
{
  "qrHeaderUUID": "string",  // Unique identifier for the QR header.
  "qrExtensionUUID": "string",  // Unique identifier for the QR extension.
  "qrAsText": "string",  // QR code represented as text.
  "qrAsImage": "string"  // QR code represented as an image (base64 encoded).
}
```

Cancel extension

URL: /api/v1/qr/{qrHeaderUUID}/active-extension

Method: DELETE

Description: Cancel active extension of hybrid payee-presented QR code

Parameters:

- qrHeaderUUID – This is the same UUID that was returned in New QR endpoint.

Extension status

URL: /api/v1/qr-extensions/{qrExtensionUUID}/status

Method: GET

Description: Get the status of a QR code extension and list of last N payments against it.

Parameters:

- nbOfTxs
- **Description**: Limits the number of requested transactions (payments).
- **Type**: integer (numberOfItems) $0 .. 10$
- **Default**: 0
- **Example**: nbOfTxs=5

Response body:

```
{
  "uuid": "string",
  "isLast": true,
  "status": "Active", // Enum: "Active", "Paid", "Expired", "Cancelled", "Replaced",
"Inactive"
  "statusDtTm": "2024-07-29T09:59:09.224Z",
  "isHeaderLocked": true,
  "ttl": {
    "length": 0,
    "units": "ss"
  },
  "payments": [
    {
      "system": "string",
      "reference": "string",
      "amount": {
        "sum": 0,
        "currency": "string"
      }
    }
  ]
}
```

Transactions

Reverse transaction

URL: /api/v1/transaction/{reference}

Method: DELETE

Description: Reverses a transaction that has already been processed.

Parameters:

- **reference –** External payment reference — the 4th pipe-delimited segment from the **callback message** (payment.reference) ,**QR Status** endpoint (payments.reference), **Extension status** endpoint (payments.reference) or from List of transactions endpoint (*miaId*).
- **Full value**:

pacs.008.001.10|2025-04-23|VICBMD2X|VICBMD2XAXXX250423463390000017890

- **Part that must be used :**

VICBMD2XAXXX250423463390000017890

Response:

- **Status Code**: 204 OK
- **Status Code**: 404 Not found
- **Status Code**: 500 Internal Error

Reconciliation

List of transactions

URL: /api/v1/reconciliation/transactions/

Method: GET

Description: Retrieve a list of transactions that occurred on a specific date, including detailed information required for reconciliation.

Query Parameters:

- *datefrom* **–** The start date of the date range for which to retrieve transactions in format YYYY-MM-DD.
- *dateto***–** The end date of the date range for which to retrieve transactions in format YYYY-MM-DD.

Request example:

GET */api/v1/reconciliation/transactions?datefrom=2024-08-01&dateto=2024-08-13*

Response body:

```
{
transactionsInfo: [
    {
      "id": " MOLDMD2XAXXX240206250390000001094", // MIA message ID (unique indentifier MIA)
      "date": "2024-08-13", // Date of the transaction
      "time": "06:41:50.3530000", // Time of the transaction
      "payerName ": "Jon Snow", // Payer's name
      "payerIdnp": "*5853",  // Payer's IDNP
      "beneficiaryIdnp": "*6067", // Beneficiary's IDNP
      "transactionType": "QR",  // Type of transaction (QR/RTP/QRREFUND)
      "transactionAmount": 1500.00,  // Transaction amount
      "transactionStatus": "Approved",//Transaction status (Pending/Rejected/Approved)
      "destinationBankName": "ARLCMD2X", // Destination bank name or BIC code
      "transactionMessage": "MIA Transfer.", // Message of transaction
      "paymentType": "IPSMIA", //Constant. For future integrations
      "miaId": "string", //The same as reference attribute in Signals,
```

```
      "creditorRef": "string",  // External payment reference. The same as in QR generation
endpoints
      "iban": "MD52VI225100000103171MDL" //The recipient IBAN account"
    }
  ]
}
```

Get Last Signal by QR Extension UUID

URL: /api/v1/signal/{qrExtensionUUID}

Method: GET

Description: Retrieve the most recent signal delivered for the specified QR extension UUID. Signals are ordered by their signalDtTm; only the latest one is returned.

Path Parameter:

- qrExtensionUUID: The UUID of the QR extension for which to fetch the latest signal. Example: aa45f600-894b-410d-9b94-65b431b099a6

Request example:

GET /api/v1/signal/aa45f600-894b-410d-9b94-65b431b099a6

Response body:

The same as in **Signals**

```
{
  "signalCode": "Payment",
  "signalDtTm": "2025-04-23T12:52:22+03:00",
  "qrHeaderUUID": "bd12049b-651d-40a0-b2bc-d6a0f7e2b7c7",
  "qrExtensionUUID": "aa45f600-894b-410d-9b94-65b431b099a6",
  "payment": {
    "system": "IPS",
    "reference": "pacs.008.001.10|2025-04-23|VICBMD2X|VICBMD2XAXXX25042346339000017890",
    "amount": {
      "sum": "3.48",
      "currency": "MDL"
    }
  }
}
```

Disclaimer: We strongly recommend integrating the POST-based /api/signals callback mechanism as the primary way to receive real-time notifications. The GET /api/v1/signal/{qrExtensionUUID} endpoint is provided only as a fallback for scenarios where implementing the callback schema is not feasible.

Signals

This section outlines the specifications for delivering signals from the bank to a partner. The partner must implement an API endpoint where the bank will send events related to the QR code lifecycle.

Certificates

In the web API configuration, it is desirable to have keys for two certificates to ensure continuity when the certificates are changed.

Certificate

VBCA.crt file

Endpoint definition

URL: /api/signals

Method: POST

Headers:

- Content-Type: application/json

Authentication:

- JWT must be included in the **request body** as a JSON string.

Request Body Structure

Format:

- **Type**: JSON string representing the JWT token.
- **Content**: The JWT must encapsulate the signals data within its payload.

JWT Payload Requirements

The JWT payload will contain a signal object that adheres to the original signal object specifications and is signed with the bank's private key. The signature must be checked using the bank's public key.

Example Payload:

```
{
    "signalCode": "Payment",
    "signalDtTm": "2025-04-23T12:52:22+03:00",
    "qrHeaderUUID": null,
    "qrExtensionUUID": "aa45f600-894b-410d-9b94-65b431b099a6",
    "payment": {
        "system": "IPS",
        "reference": "pacs.008.001.10|2025-04-
23|VICBMD2X|VICBMD2XAXXX25042346339000017890",
        "amount": {
            "sum": "3.48",
            "currency": "MDL"
        }
    }
}
```

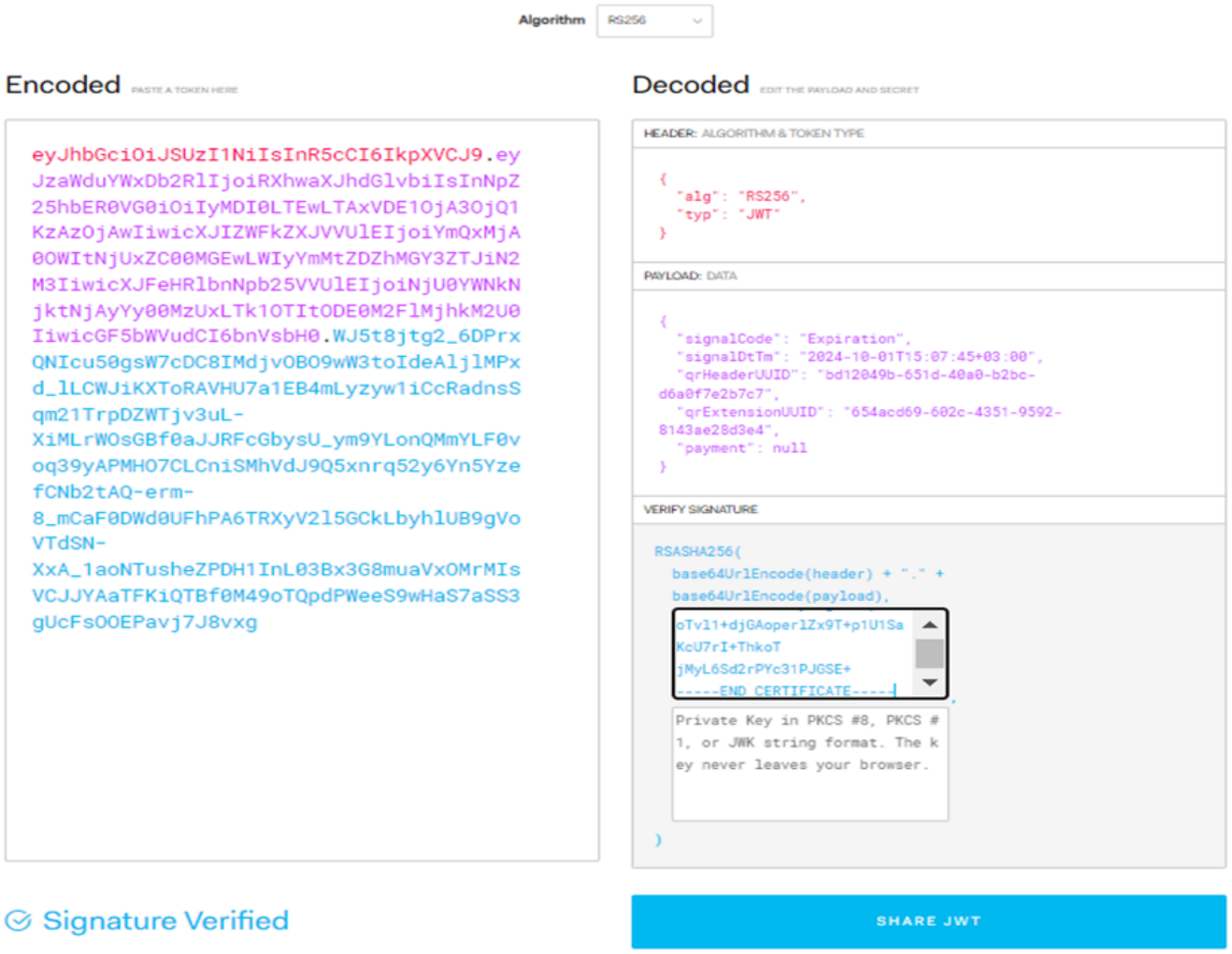
Example cURL Request:

```
curl --location '<https://test-demo-ips-signal-api.ad.victoriabank.md/api/signals>' \
     --header 'Content-Type: application/json' \
     --data '"eyJhbGciOiJSUzI1NiIsInR..."'
```

Response:

- **Status Code**: 204 OK
- **Status Code**: 400 Bad Request
- **Status Code**: 404 Not found
- **Status Code**: 500 Internal Error

JWT.IO Example



Signal Object Properties

- **signalCode** (required):
  - **Type**: string
  - **Enum**: "Payment", "Expiration", "Inactivation"
  - **Description**: Indicates the type of signal being sent.
- **signalDtTm** (required):

    - **Type**: string (<date-time>)
    - **Description**: The date and time the signal was generated.
- **qrHeaderUUID**:
    - **Type**: string (uuid)
    - **Pattern**: ^$0-9a-f${8}-$0-9a-f${4}-$0-9a-f${4}-$0-9a-f${4}-$0-9a-f${12}$
    - **Description**: Unique identifier for the QR header. (Optional if qrExtensionUUID is provided)
- **qrExtensionUUID**:
    - **Type**: string (uuid)
    - **Pattern**: ^$0-9a-f${8}-$0-9a-f${4}-$0-9a-f${4}-$0-9a-f${4}-$0-9a-f${12}$
    - **Description**: Unique identifier for the QR extension. (Optional if qrHeaderUUID is provided)
- **payment** (optional):
    - **Type**: object (paymentOut)
    - **Description**: Information about the payment related to the signal.
    - **Properties**:
        - **system** (required):
            - **Type**: string
            - **Enum**: paymentSystemVirtual, paymentSystemDefault, paymentSystemExtra
            - **Description**: The payment system used.
        - **reference** (required):
            - **Type**: string (paySysReference)
            - **Pattern**: ^(pacs\.008\.001\.\d{2}\|20(2$4-9$|$3-9$\\d{1})\d{4}\|$0-9${10}\-$0-9${4})$
            - **Description**: A unique payment system reference.
        - **amount** (optional):
            - **Type**: object (money)
            - **Description**: Details of the payment amount.
            - **Properties**:
                - **sum**: string
                - **currency**: string
        - **dtTm** (optional):
            - **Type**: string (<date-time>)
            - **Description**: Date and time of the payment.
        - **signalIntermediary** (optional):
            - **Type**: object (agent)
            - **Description**: Intermediary details for the signal.

Note on Extracting RRN (Retrieval Reference Number):

To extract the RRN, follow this rule:

- Take the value of the "reference" attribute.
- Split the string using the pipe symbol (|) as delimiter.
- From the 4th segment (index 3), extract the last 12 characters.
- If the 4th segment is shorter than 12 characters, use it as-is.

Example:

"reference": "pacs.008.001.10|2025-04-23|VICBMD2X|VICBMD2XAXXX250423463**390000017890**"

Disclaimer:

RRN is not guaranteed to be unique across all systems or timeframes. It should be treated as a transaction identifier for reference purposes only, and not relied upon as a strict unique key in processing or reconciliation.

Integration recommendations

QR Payment integration

Database preparation

Ensure that the order tracking system supports storing QR-related metadata. To achieve this, one of the following options should be implemented:

Option A:

Add two new columns to the existing orders table:

- `qrExtensionUUID` – the unique identifier of the generated QR extension.
- `reference` (optional) – a field for storing the payment reference from the received signal.

Option B:

Create a separate table to associate the order ID with the `qrExtensionUUID` and the reference.

QR Code Generation

When generating a QR code using the MIA Business API(`POST: /api/v1/qr`), store the returned `qrExtensionUUID` in the newly added column of the orders table, or in the separate mapping table created in previous step.

Signal Processing

Upon receiving a payment signal from MIA Business, proceed as follows:

- Use the `qrExtensionUUID` from the signal to identify the corresponding order in your system.
- (Optional) Extract and store the reference field from the signal for future use:

```
"reference": "pacs.008.001.10|2025-04-23|VICBMD2X|VICBMD2XAXXX250423463390000017890"
```

Part to should be saved for future (`reference`) is: **VICBMD2XAXXX250423463390000017890**

(Optional) Retrieve Additional Transaction Details

With the reference obtained, you may query the following endpoint to get detailed information about the transaction:

`GET: /api/v1/reconciliation/transactions`

Alternative to Saving `reference` Field

If you prefer not to store the `reference` in your database, it can still be retrieved dynamically from one of the following endpoints:

- `GET: /api/v1/qr-extensions/{qrExtensionUUID}/status`
- `GET: /api/v1/signal/{qrExtensionUUID}`

Once retrieved, you can proceed with **"Retrieve Additional Transaction Details"** step.

Summary

All steps of **Payment integration** can be summized in following steps:

Step	Action
1	Prepare database schema to store `qrExtensionUUID` (and optionally `reference`).
2	Store `qrExtensionUUID` upon QR generation.
3	On payment signal, identify order by `qrExtensionUUID` and optionally save `reference`.
4	Retrieve transaction details using the `reference`.

Payment simulator

URL: https://test-ipspj-demopay.victoriabank.md/api/pay/

Method: POST

Description: This API enables payment simulation based on the *qrHeaderUUID*, without requiring any bank or financial institution applications.

Swagger: https://test-ipspj-demopay.victoriabank.md/swagger/index.html

Body:

```
{
  "qrHeaderUUID": "string"
}
```

Response

- **202 Accepted** – Transaction successful.
- **400 Bad Request** – Unsuccessful.

Response Body

In this response body, the API indicates a client error by including a plain-text message describing the specific issue. For example, the text "EQV|Target QR code does not exist" conveys that the requested QR code resource was not found. This error message allows the client to understand the cause of the 400 Bad Request status and take corrective action (e.g., checking or providing a valid QR code).

History

Version 2.0.5

- Payment simulator compartment was added.

Version 2.0.6

- Added the **List of transactions** endpoint.

Version 2.0.7

- **Payment Simulator** URL was corrected.

Version 2.0.8

- Added a response body description for the payment simulator.
- The successful response code in the payment simulator was changed from 200 to 202.

Version 2.0.9

- Enhanced **Reverse Transaction** endpoint definition.
- Expanded **Signals** request-body definition: added the *payment* attribute.

Version 2.0.10

- Changes in the **List of transactions** endpoint
    - Root key corrected from "transactions" to "transactionsInfo".
    - Key "payerName " was corrected by removing the trailing space; now properly written as "payerName".
    - Transaction type description in the comment was expanded to include "QRREFUND" as a possible value.
    - New attribute "miaId" was added to represent the MIA reference, aligning it with the reference attribute used in Signals.
- A new explanatory note was added to the **Signals** compartment describing how to extract the RRN (Retrieval Reference Number) from the `reference` field. The note includes parsing logic, an example, and a disclaimer stating that the RRN may not be unique.

Version 2.0.11

- Added the new GET */api/v1/signals/{qrExtensionUUID}* endpoint to allow clients to retrieve the most recent signal for a given QR extension UUID.

Version 2.0.12

- Updated the public certificate in **Certificates** compartment for client JWT verification

Version 2.0.13

- Updated the public certificate in Certificates compartment for client JWT verification
- Removed the GET */api/v1/transactionstate/{qrExtensionUUID}* endpoint as it was deprecated and is no longer in use

Version 2.0.14

- The endpoint URL has been changed from `/api/v1/signals/{qrExtensionUUID}` to `/api/v1/signal/{qrExtensionUUID}`.

Version 2.0.15

- Added detailed information about `qrType` and `amountType`
- Added recommendation about integration with **VB MIA API** via `QR Payment`

Version 2.0.16

- Introduced the `iban` field (Recipient IBAN) to the transaction response for the `/api/v1/reconciliation/transactions/` endpoint.

Version 2.0.17

- Attribute `creditorRef` has been added to the **List of transactions** endpoint.

Version 2.0.18

- In the endpoint `/api/v1/qr`, the attribute `pmtContext` value was changed from **0** (number) to **O** (letter).